

课程笔记: Learning while you sleep: Beyond memory to dreaming

SSS & Codex

2026 年 6 月 29 日



视频作者/频道: AI Native Dev

发布日期: 2026-06-22

视频时长: 31:58

视频链接: <https://www.youtube.com/watch?v=tTcxVv8HHNw>

PDF 制作 Skill: <https://github.com/wdkns/wdkns-skills>

目录

1 为什么模型智能需要上下文工程	2
1.1 演讲者与应用 AI 场景	2
1.2 核心问题：把通用智能翻译成产品表现	3
1.3 上下文工程的乘法效应	4
1.4 常见失败：智能体缺少三类关键上下文	4
1.5 本课路线：从上下文原语走向持续学习闭环	5
1.6 本章小结	5
2 从 CLAUDE.md 到技能：上下文原语的演进	5
2.1 CLAUDE.md：最早跑通的上下文文件	6
2.2 Markdown 记忆：人类可读，智能体可写	6
2.3 Autonomous memory tools：把读写判断交给智能体	6
2.4 Skills 与渐进披露：先看书脊，再读正文	7
2.5 File-system memory stores：用普通文件和搜索承载记忆	8
2.6 三条早期经验：Markdown、索引规模、自主写入	8
2.7 本章小结	9
3 生产级记忆系统的四个护栏	9
3.1 规模化风险：共享记忆会把小错误放大成系统错误	10
3.2 Versioning：每次更新都要能追溯与回滚	10
3.3 Concurrency：用哈希检查避免覆盖他人的更新	10
3.4 Permissions：不同记忆层级需要不同读写权	12
3.5 Portability：把记忆做成可复用资产	12
3.6 Production outcomes：护栏的目标是让自治产生收益	13
3.7 本章小结	13
4 In-band 记忆的边界：为什么单个会话看不到系统性错误	14
4.1 In-band memory：读写发生在同一个活跃会话内	14
4.2 资源分裂：当前任务和未来学习争用同一预算	15
4.3 可见性限制：单个会话看不到跨会话模式	15
4.4 学校类比：学生、教师和校长为什么分工	16
4.5 Freshness：过期记忆需要单独检查	16
4.6 本章小结	16
5 Dreaming：异步整理经验的二阶记忆过程	17
5.1 三层结构：context、memory processes 与 dreaming	17
5.2 基本流程：从记忆库与 transcripts 到分析智能体	18
5.3 输出形式：带例证和流行度统计的记忆变更提案	19
5.4 三个直观案例：课程缺口、工具配置与组织偏好	19
5.5 Orchestrator 与 subject agents：分布式审阅和聚合	20
5.6 Human or policy acceptance：接受、拒绝或细化提案	21
5.7 本章小结	21

6 部署边界：成本、权限与数据库化的回归	22
6.1 Memory 与 dreaming 并行：快改进与广视野	22
6.2 成本论证：额外整理资源如何减少重复浪费	23
6.3 最终演讲总结：从简单上下文到闭环学习	24
6.4 Enterprise implementation: managed memory 与 dreaming APIs	24
6.5 Permission mirroring: 只选择匹配权限边界的 transcripts	25
6.6 Database question: 把成熟原语沉入 harness	26
6.7 本章小结	27
7 总结与延伸：让智能体在睡眠中学习	27
7.1 从演讲总结得到的建设顺序	27
7.2 分阶段 Builder Checklist	28
7.3 Q&A 的三个落地补充	30
7.4 实践含义：什么时候该加 dreaming	31
7.5 开放问题：这仍是快速演化的研究与产品领域	31
7.6 读者自检清单	32
7.7 本章小结	32

1 为什么模型智能需要上下文工程

模型智能要变成持久、可扩展、有用的产品，关键条件是上下文工程 (context engineering)：产品必须把组织环境、任务目标、代码库结构、用户偏好和历史经验交给智能体。更强的基础模型会提高起点，可实际表现仍取决于它是否知道“在这里怎样算做对”。Lamis Mukta 在开场把问题压缩成一句工程判断：raw model intelligence 要进入真实产品，必须靠上下文把通用能力翻译成具体环境里的行动能力。¹

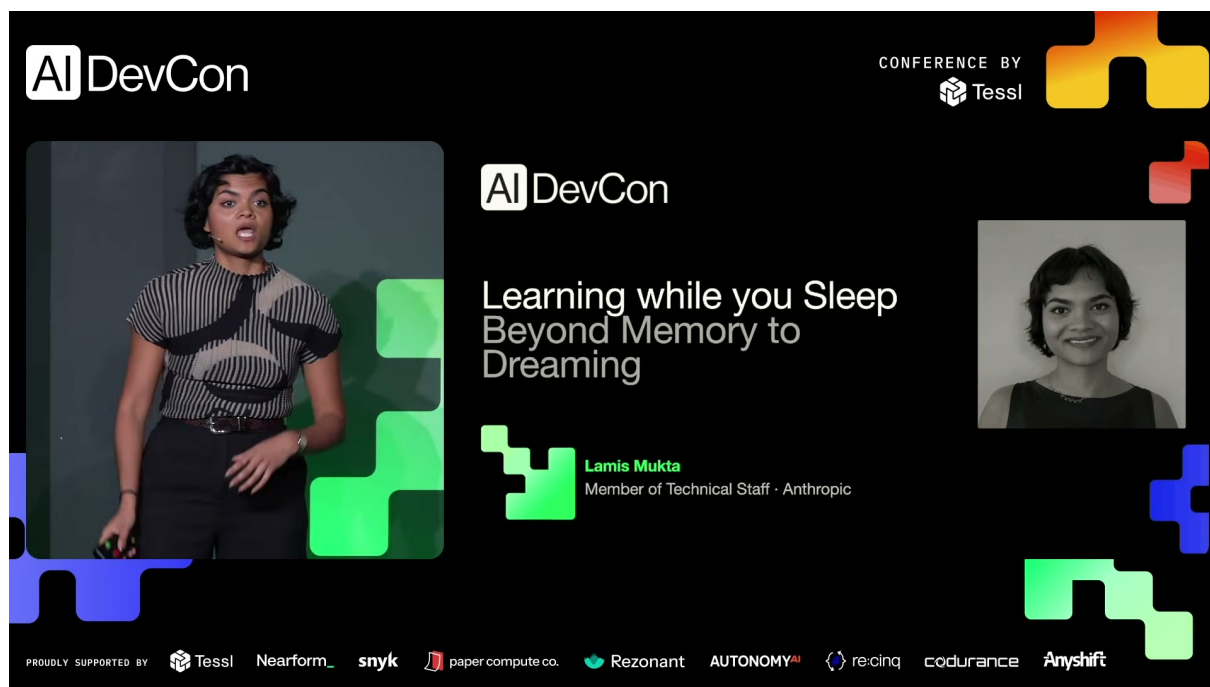


图 1：标题页明确了全篇主题：从上下文工程和记忆系统出发，走向用 dreaming 巩固经验的持续学习闭环。²

1.1 演讲者与应用 AI 场景

这场演讲来自 Anthropic applied AI 团队的 Lamis Mukta。她所处的位置很特殊：团队连接 research、product 和 go-to-market，同时也直接面对正在构建 frontier agents 的客户与创业团队。³这意味着演讲的视角偏工程落地：讨论重点从模型排行榜上的单点能力，推进到模型进入产品、组织和长期工作流之后的真实表现。

为什么“应用 AI”视角重要

研究团队常问“模型能不能做到某件事”，产品团队还要追问“它能不能在用户的真实环境里反复做到”。二者的差别像驾驶技术和城市地图：驾驶技术再强，缺少道路、规则、目的地和实时路况，也很难稳定到达终点。上下文工程提供的正是这张“任务地图”。

这类场景给模型施加了三重压力。第一，任务有组织边界，例如代码库结构、内部术语和审批规则。第二，用户有个人偏好，例如写作风格、工具选择和交付格式。第三，产品希望系统越用越

¹来源区间：00:01:24–00:01:41。

²视频画面时间区间：00:01:24–00:01:41。

³来源区间：00:00:39–00:01:14。

顺手，这要求智能体从前一次失败中留下可复用经验，而非每次从零开始。

1.2 核心问题：把通用智能翻译成产品表现

演讲的核心问题可以写成一个简单的乘法模型：

$$Q_{\text{product}} \approx I_{\text{model}} \times F_{\text{context}}$$

其中：

- Q_{product} ：产品中的实际任务质量，包括准确性、稳定性、成本和用户体验。
- I_{model} ：基础模型的通用智能，也就是没有特定产品上下文时的能力底座。
- F_{context} ：上下文适配度，表示系统提供给模型的环境知识、任务约束和历史经验是否足够贴合当前任务。

这个公式不追求精确计量，它的教学价值在于提醒读者：当 F_{context} 很低时， I_{model} 的提升会被真实环境的缺口吞掉。新模型可以更会推理，可它仍然不会天然知道某个公司的目录结构、某个团队的发布流程、某位用户讨厌的措辞，以及上周某个工具调用为何失败。

本章主结论

上下文工程把模型智能从“通用能力”转换成“环境内能力”。如果没有这层转换，智能体的表现很容易停留在会说、会猜、会临场补救，却无法稳定沉淀为产品能力。

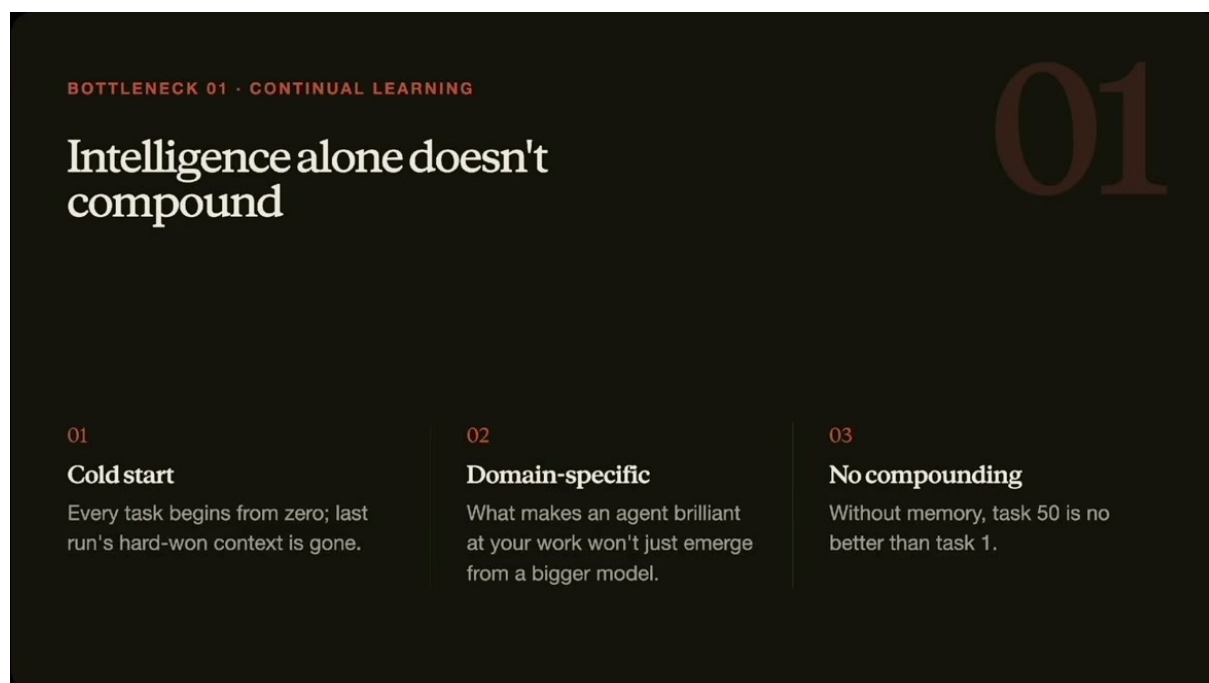


图 2：模型智能本身无法自动复利；冷启动、领域知识和经验沉淀缺口会让智能体在真实产品中失去稳定性。⁴

⁴视频画面时间区间：00:02:29–00:03:38。

1.3 上下文工程的乘法效应

Lamis 强调，Anthropic 会不断发布更聪明的模型，可当模型被部署到用户自己的 agents、environments 和 organizations 中时，intelligence alone does not compound。⁵ 这句话的关键含义是：模型进步提供可用能力，上下文工程决定这份能力能否在具体组织里复利增长。

复利增长来自两类积累。第一类是静态积累：团队把“如何在这个环境中做事”写成可读、可检索、可维护的上下文，例如项目说明、代码约定、工具说明和用户偏好。第二类是动态积累：系统把任务中的失败、修正和偏好反馈变成下一次可用的记忆。前者像给新人一份入职手册，后者像团队每周复盘后更新操作规程。

直觉类比：模型像引擎，上下文像传动系统

更强的模型像马力更大的引擎。上下文工程像传动系统、轮胎和路面信息，它决定马力是否真的转化为抓地力。没有上下文时，模型可能很强，却把力气花在猜目录、猜偏好、猜工具约束上。

这也是为什么 context engineering 是产品团队的主杠杆之一。训练一个 frontier model 通常超出产品团队控制范围，可产品团队可以设计更好的上下文文件、更明确的技能、更可靠的记忆库，以及更可审计的更新流程。随着基础模型继续增强，已经整理好的上下文资产会继续放大新模型的能力。

1.4 常见失败：智能体缺少三类关键上下文

演讲列出的失败模式非常具体：智能体找不到代码库方向、不理解用户偏好、无法从上一次错误中变得更好。⁶ 这三类问题分别对应三种上下文缺口。

- 代码库方向缺口：智能体不知道模块边界、测试入口、生成文件和禁止修改区域。它可能读很多文件，仍然抓不到项目结构。
- 用户偏好缺口：智能体不知道用户希望的语言、交付形态、细节密度、风险口径和审美要求。它可能给出“正确”答案，却不符合用户真正要的工作方式。
- 经验沉淀缺口：智能体重复犯同一类错误，例如反复调用错误工具、反复忽略某个验证步骤、反复写出不合项目风格的输出。

常见误解：把失败归咎于模型笨

很多产品问题看起来像模型能力不足，实际根因常常是上下文供应不足。一个智能体如果没有项目地图、用户偏好和历史复盘，就像新员工每天被派去处理复杂任务，却不给入职资料、不给工作规范、也不给昨日事故记录。

这些失败也说明，continual learning（持续学习）在这里并不等同于实时训练模型参数。演讲讨论的是系统层面的持续学习：通过上下文、记忆和后续的 dreaming 流程，让智能体在未来任务中带着更好的环境知识出发。

⁵来源区间：00:02:29–00:03:16。

⁶来源区间：00:03:16–00:03:38。

1.5 本课路线：从上下文原语走向持续学习闭环

第一部分建立问题：模型智能需要上下文才能在产品中复利。第二部分回顾已经证明有效的上下文原语，包括 CLAUDE.md、memory tools、skills 和 file-system memory stores。随后课程会进入生产级记忆系统的护栏，再解释 in-band memory 的边界，最后引出 dreaming 这一会话外、批处理式的记忆整理过程。

这个路线本身就是一个金字塔结构：先回答“为什么需要上下文”，再回答“已有原语如何工作”，接着回答“规模化时怎样治理”，最后回答“系统如何跨会话学习”。读者若只记住一件事，应记住：智能体产品的长期质量来自模型智能与上下文系统的共同演化。

1.6 本章小结

本章给出全篇的起点：raw model intelligence 只有进入具体环境，才会成为稳定的 product performance。上下文工程提供环境知识、用户偏好和历史经验，使更强模型的能力能够被组织吸收并复用。

后续章节会沿着这一问题继续推进：先看 CLAUDE.md、技能和文件系统记忆如何出现，再看这些机制进入生产后为什么需要版本、并发、权限、可移植性和 dreaming。

2 从 CLAUDE.md 到技能：上下文原语的演进

上下文工程过去一年的演进说明了一个朴素规律：简单、可读、可被人 and 智能体共同维护的 artifacts 往往先跑起来；当它们变长、变多、变复杂之后，系统才需要渐进披露 (progressive disclosure)、自主记忆工具和文件系统式记忆库来控制规模。Lamis 用 Anthropic 的一句工程原则概括这一阶段：do the simple thing that works。⁷

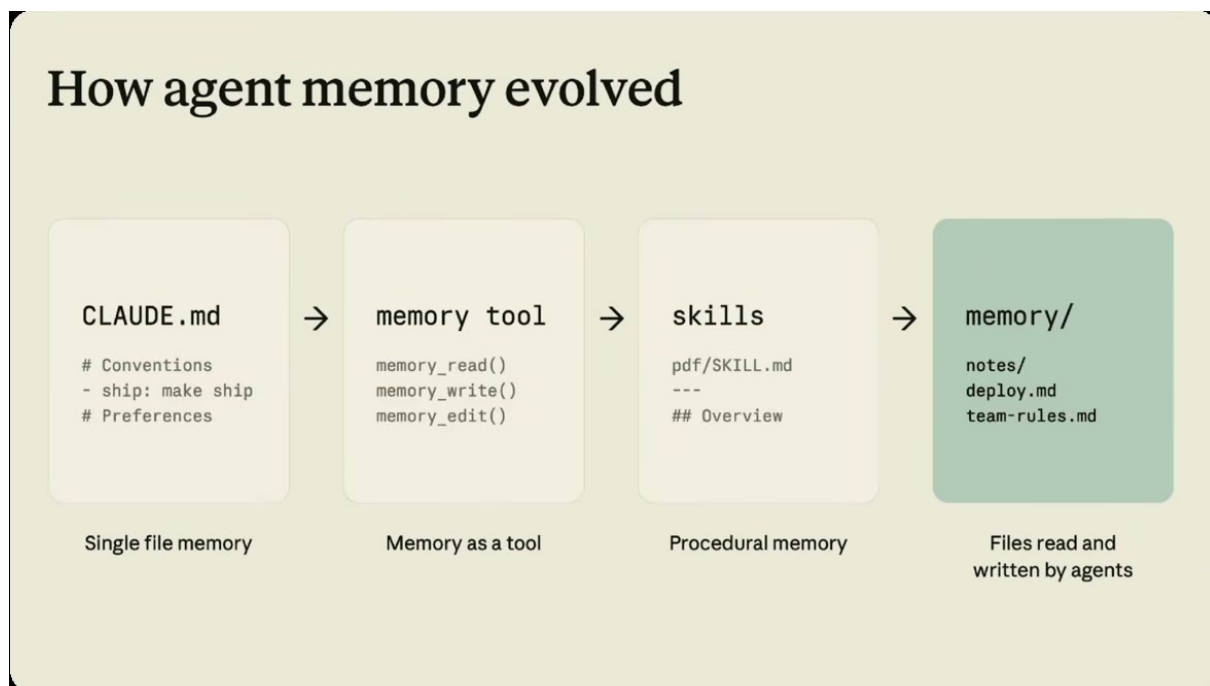


图 3: 记忆原语从单个 CLAUDE.md 文件，发展到 memory tools、skills，再到可读写、可搜索的文件系统式记忆库。⁸

⁷来源区间：00:03:42–00:03:55。

2.1 CLAUDE.md：最早跑通的上下文文件

CLAUDE.md 的价值在于低技术门槛和高指导密度。它只是一个 Markdown 文件，却能在会话开始时把代码库方向、组织背景和用户偏好注入模型上下文，从而显著改善智能体的导航和执行。⁹

CLAUDE.md 的核心机制

CLAUDE.md 把隐性的工作约定显式化。它让智能体在动手前先知道项目边界、用户偏好和常用流程，相当于给每次会话发一张“当前环境说明卡”。

它“unreasonably effective”的原因很直接。Markdown 同时适合人读、适合人写、适合智能体读取和改写。它无需复杂数据库或检索系统，就能承载相当多的实践知识。早期上下文工程能迅速扩散，也正因为这种形式足够普通：开发者已经会用文件，智能体也已经会读文件。

不过这个原语很快遇到增长压力。重要偏好、项目说明和流程经验越积越多，单个会话开始时全部注入会吞掉上下文窗口，也会让模型在无关信息中分心。¹⁰ 文件越长，越需要新的加载策略。

规模化风险：长上下文文件会变成噪声源

上下文越多并不自动更好。若系统把全部信息一次性塞进会话，智能体会把注意力浪费在低相关内容上。真正的目标是“相关内容及时出现”，而非“所有内容同时出现”。

2.2 Markdown 记忆：人类可读，智能体可写

从 CLAUDE.md 得到的第一个教训是：Markdown memory 很适合做记忆载体。Lamis 强调，一个简单、人类可读、智能体可写、用户也能改的 Markdown 文件，对引导智能体非常有效。¹¹

这背后有一个工程上的互补关系。人类擅长判断一条经验是否值得保留，智能体擅长在任务过程中发现自己需要哪些说明。Markdown 把两者放在同一种介质里：人可以审阅和修订，智能体可以读取和提出更新，普通搜索工具也能快速定位。

为什么文件比“黑盒记忆”更适合作为早期原语

文件系统有天然的可见性。用户能打开、搜索、比较、提交版本，也能在代码审查里看到变更。黑盒记忆可能更自动，却很容易让团队失去“这条记忆从哪里来、谁写的、为什么影响行为”的审计能力。

2.3 Autonomous memory tools：把读写判断交给智能体

第二个原语是 memory tools。它把问题从“会话开始时注入什么”推进到“会话进行中由智能体决定何时读、写、更新”。Lamis 将这种方式称为 in-band：记忆动作发生在当前任务会话内部，智能体在解决任务的同时判断哪些经验值得读取或沉淀。¹²

这一步的重要性在于 autonomy（自主性）。如果所有记忆更新都依赖用户手动记录，系统会漏掉大量细碎经验；如果智能体能够主动读取和写入，它就有机会把任务现场的失败原因、工具选择和用户反馈转化为未来上下文。

⁸ 视频画面时间区间：00:03:55–00:08:20。

⁹ 来源区间：00:03:57–00:04:25。

¹⁰ 来源区间：00:04:25–00:04:45。

¹¹ 来源区间：00:04:46–00:05:02。

¹² 来源区间：00:05:02–00:05:36。

可以用一个轻量公式理解 in-band 记忆的压力：

$$C_{\text{session}} = C_{\text{task}} + C_{\text{mem}}$$

其中：

- C_{session} ：当前会话的总容量，包括上下文窗口、时间、token 和用户耐心。
- C_{task} ：用于完成当前任务的容量。
- C_{mem} ：用于读取、判断、更新记忆的容量。

这个公式也提前埋下后文的问题：会话内记忆能快速改善下一次任务，但它会占用当前会话资源，并且很难看见跨会话、跨智能体的系统性失败。

2.4 Skills 与渐进披露：先看书脊，再读正文

Skills（技能）解决的是不断增长的上下文问题。它们把流程知识打包成带有 front matter 的程序性说明：智能体先扫描顶部几句话，判断技能是否相关；相关时才加载完整内容。¹³ 这就是 progressive disclosure。

$$C_{\text{loaded}} = C_{\text{front}} + \mathbf{1}_{\text{relevant}} C_{\text{body}}$$

其中：

- C_{loaded} ：实际载入会话的上下文量。
- C_{front} ：技能 front matter 的简短索引成本。
- $\mathbf{1}_{\text{relevant}}$ ：相关性指示，相关时取 1，无关时取 0。
- C_{body} ：技能正文中详细流程、脚本、模板和约束的上下文成本。

Lamis 用书架类比解释这一点：人不需要把七年法语课内容一直装在脑子里，只需要看到“法语词典”这本书在架上；当有人开始说法语时，再把相关书拿下来读。¹⁴

技能的教学价值

技能把“流程知识”从一次性提示中分离出来。它适合那些有端到端做法、有固定检查点、有工具或模板依赖的任务，例如生成 PDF、做安全扫描、处理字幕、操作浏览器。

技能的限制也很清楚：它仍然需要人和智能体共同策划。团队要判断哪些流程值得封装成技能，技能名称和 front matter 是否足够可发现，正文是否足够可靠。换句话说，skills 降低了上下文加载成本，同时引入了技能库治理问题。¹⁵

¹³ 来源区间：00:05:53–00:06:41。

¹⁴ 来源区间：00:06:41–00:07:13。

¹⁵ 来源区间：00:07:17–00:07:31。

2.5 File-system memory stores: 用普通文件和搜索承载记忆

演讲把当前 memory systems 的最佳实践描述为 file-system memory stores: 用目录组织 Markdown 记忆, 让智能体使用普通文件系统工具, 例如 search、bash、grep, 去索引和定位相关内容。

¹⁶

这种设计吸收了前面几步的教训。Markdown 保持可读和可写; 目录结构提供分层; 搜索工具提供按需检索; 智能体自主选择读写时机。它把“记忆”从一个神秘功能还原成可检查的工程资产: 一组文件、一套索引策略、一组权限和更新约定。

文件系统记忆库的直觉

把 memory store 想成团队知识库, 而智能体像一名能用搜索工具的新同事。它先检索相关目录, 再打开少量文件。这样做既保留了大规模记忆, 又避免把整座知识库一次性塞进会话。

这一步也把上下文工程带入后续章节的问题: 一旦 memory store 变成多人、多智能体、长期运行的共享资产, 就必须面对版本管理、并发控制、权限、可移植性和过期记忆等生产级风险。

2.6 三条早期经验: Markdown、索引规模、自主写入

Lamis 在这一段最后给出三条总结: Markdown is great for reading memories; 允许记忆变大, 但要给智能体快速索引和搜索相关内容的工具; 给智能体写入记忆的 autonomy。¹⁷



图 4: 早期经验可以压缩成三点: 用文件格式承载记忆, 通过搜索渐进读取, 并让智能体在合适时机自主写入。¹⁸

¹⁶来源区间: 00:07:35–00:08:20。

¹⁷来源区间: 00:08:22–00:08:41。

¹⁸视频画面时间区间: 00:08:22–00:08:41。

Section 2 的压缩结论

上下文原语的演进路线是：先用简单 Markdown 文件把环境知识显式化，再用 memory tools 让智能体在会话中管理记忆，然后用 skills 做渐进披露，最后用 file-system memory stores 承载可搜索、可增长、可审计的长期记忆。

这三条经验构成一个可操作的起步方案。小团队可以从一份 CLAUDE.md 或等价上下文文件开始；流程稳定后，把复杂步骤沉淀为 skills；经验积累到一定规模后，再把记忆拆成文件系统式的 memory store，并给智能体足够好的搜索入口。

2.7 本章小结

本章解释了上下文工程的早期原语如何逐步演进。CLAUDE.md 证明简单 Markdown 文件能显著引导智能体；memory tools 把读写时机交给智能体；skills 通过 progressive disclosure 控制上下文加载；file-system memory stores 则把长期记忆放进可搜索、可审计的文件结构中。

下一章会处理这条路线的现实代价：当记忆从个人辅助工具变成生产系统，共享写入、错误传播、权限边界、过期内容和提示注入风险都会出现，单靠“让智能体自己写文件”已经不够。

3 生产级记忆系统的四个护栏

自主记忆要进入生产环境，核心结论很直接：让智能体自己读写记忆只解决了“能学习”的问题，版本管理 (versioning)、并发控制 (concurrency)、权限 (permissions) 和可移植性 (portability) 才决定这种学习会不会污染全局、覆盖他人更新、泄露边界，或被锁死在单一产品里。演讲者在 00:08:42–00:10:27 先承认文件系统式记忆能带来“持续学习的感觉”，随后立刻指出多智能体、长时间运行、复杂代码库会放大共享记忆的风险。

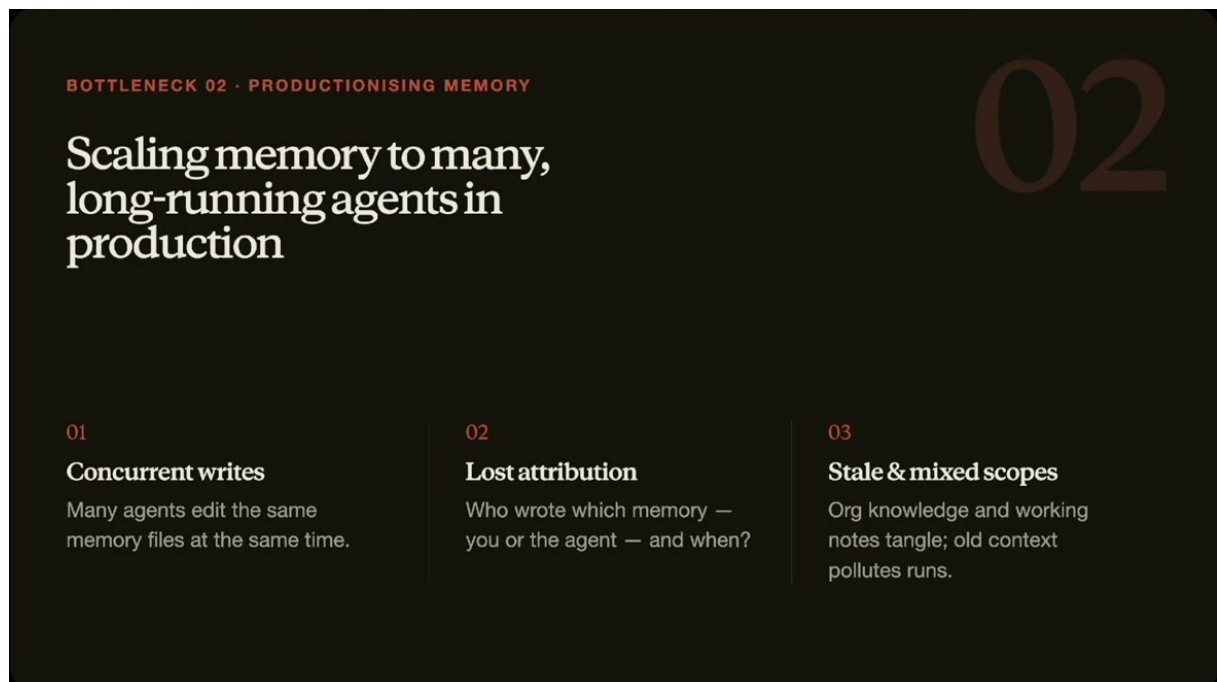


图 5: 当记忆系统扩展到许多长时间运行的智能体时，并发写入、归因丢失和过期或混杂上下文会成为生产级风险。¹⁹

生产级记忆的判断标准

生产级记忆系统的判断标准不应停在“智能体能写入一条经验”。更严格的标准是：每次写入都能追溯来源、检测冲突、服从权限边界，并且能被未来的产品界面或执行框架复用。

3.1 规模化风险：共享记忆会把小错误放大成系统错误

这一节的起点是规模化风险：当许多智能体同时依赖同一套 memory store（记忆库）时，一条错误记忆会像污染了公共路标。单个智能体在本地会话里写错一条偏好，影响可能有限；若它直接改写组织级上下文，所有后续智能体都会沿着同一条错误提示行动。演讲者给出的具体风险包括：多个智能体同时写同一份记忆文件、单个智能体把局部问题写进组织级上下文、人类和智能体一起维护记忆时难以追踪变更、记忆过期，以及 prompt injection（提示注入）把恶意指令写入未来上下文。

20

这里的“护栏”（guardrails）指工程上可检查的约束，而非抽象的安全口号。它们像数据库事务、代码评审和访问控制的组合：允许系统继续自动化，又把自动化造成的连锁损害限制在可发现、可回滚、可审计的范围内。

共享记忆的放大效应

最危险的生产误解是把“智能体能自我改进”理解成“智能体可以随意改写共享记忆”。在共享系统里，记忆是一种基础设施资产，错误更新会被后续检索、提示拼接和任务执行反复放大。

3.2 Versioning：每次更新都要能追溯与回滚

第一个护栏是 versioning（版本管理）：记忆系统必须保存版本，记录更新来自哪个上下文、哪个会话、哪段 transcript（转录记录），以及由哪个智能体或人类执行。²¹ 这件事的价值不只在“有历史记录”。它真正提供的是责任链和恢复路径：当某条记忆让任务质量下降，系统可以回到旧版本，也可以查明这条更新依据的是一次特殊失败、一次通用规律，还是一次被污染的会话。

可以把版本管理想成代码仓库里的提交历史。没有提交历史时，代码出错只能靠猜；有了提交历史，团队能看到谁改了什么、为什么改、改动对应哪个问题。记忆系统也一样：每条经验都应该带着来源证据进入库中，否则“经验”很快会变成无法质询的传闻。

版本管理回答的问题

版本管理至少应回答四个问题：这条记忆是什么时候产生的；它基于哪次会话或哪组证据；谁或哪个智能体写入了它；如果它造成负面效果，系统如何回滚。缺少任一项，后续审计都会变成高成本侦探工作。

3.3 Concurrency：用哈希检查避免覆盖他人的更新

第二个护栏是 concurrency（并发控制）。演讲者描述的做法属于乐观并发控制：智能体准备编辑前先记录当前记忆的 hash（哈希值），写入前再取一次哈希；如果两次哈希一致，说明这段时间

¹⁹ 视频画面时间区间：00:08:42–00:10:27。

²⁰ Source: 00:09:33–00:10:27.

²¹ Source: 00:10:42–00:11:17.

里记忆没有被别人改动，可以提交；如果不一致，说明中途发生了更新，智能体需要重新加载、协调差异，再尝试提交。²²

这个机制可以用一个轻量公式表示。系统只允许在以下条件成立时提交：

$$H(M_{\text{before}}) = H(M_{\text{current}})$$

符号说明：

- M_{before} ：智能体开始草拟更新时看到的记忆状态。
- M_{current} ：智能体准备提交时系统中的当前记忆状态。
- $H(M)$ ：对记忆状态 M 计算出的哈希值，也就是内容指纹。
- 等号成立：提交期间没有检测到其他写入，更新可以继续进入下一步。
- 等号不成立：记忆已经变化，智能体必须重新读取、合并差异，并再次尝试。

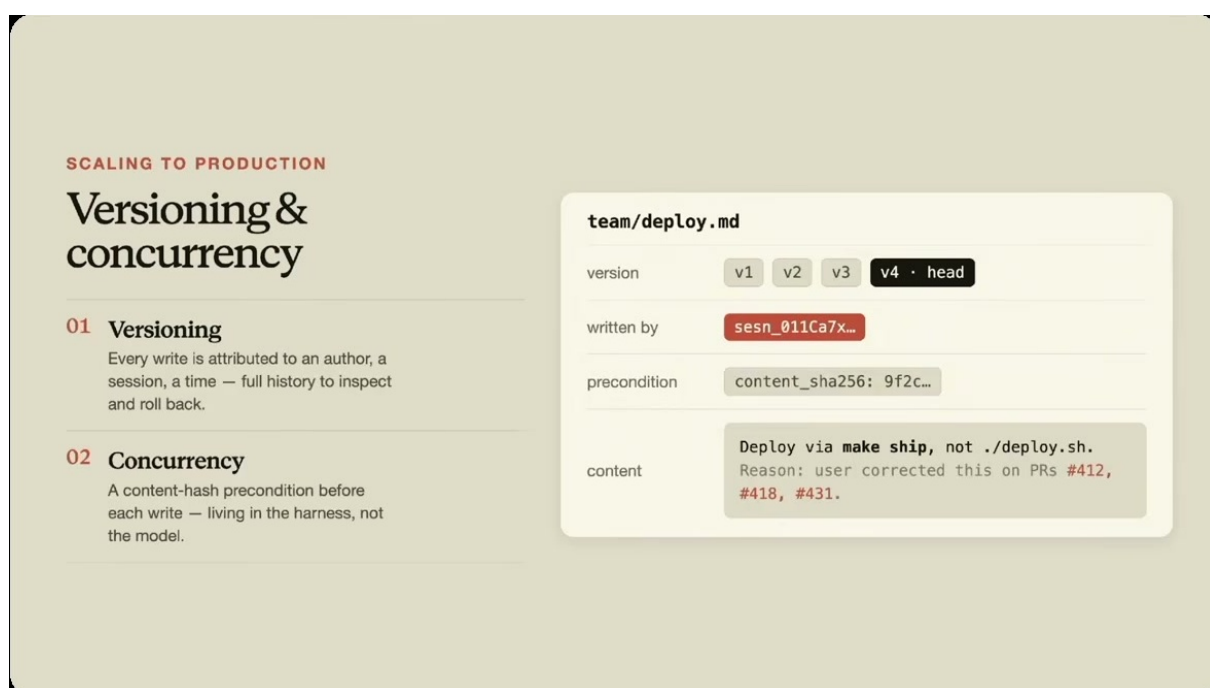


图 6：版本管理和哈希检查把共享记忆写入变成可追溯、可检测冲突的乐观并发流程。²³

哈希检查的职责

哈希并发控制的教学重点是“检测变化”，并不负责“判断谁的内容更好”。它只回答一个问题：智能体基于旧版本草拟的更新，是否仍然适用于当前版本。内容质量还需要证据、评审或策略层判断。

这种设计把共享记忆从“谁最后写入谁获胜”的脆弱模型，改造成“提交前必须确认底稿仍然有效”的可治理模型。直观类比是多人同时编辑同一份路线图：一个人拿着旧地图画新路，如果期间别人已经改了桥的位置，系统会要求他先看新地图再落笔。

²²Source: 00:11:17–00:12:05.

²³视频画面时间区间：00:10:42–00:12:05。

3.4 Permissions: 不同记忆层级需要不同读写权

第三个护栏是 permissions (权限)。演讲者把记忆从组织级知识一路讲到智能体 scratchpad (临时工作记忆): 组织目标、代码库关键原则、团队范围经验、特定任务上下文、单个智能体的工作草稿, 这些信息的价值和风险完全不同。²⁴ 因此, 权限不能只区分“能访问”和“不能访问”, 还要区分读、写、建议更新、审批更新等动作。

组织级记忆通常应该更接近只读资源, 因为它会影响许多智能体; 团队或项目级记忆可以允许更小范围的编辑; 单个智能体的 scratchpad 可以给更宽松的写入权限。这个层级像办公楼的门禁: 大厅公告可以被所有员工读取, 财务室需要更严格的进入条件, 个人笔记本则由个人频繁修改。

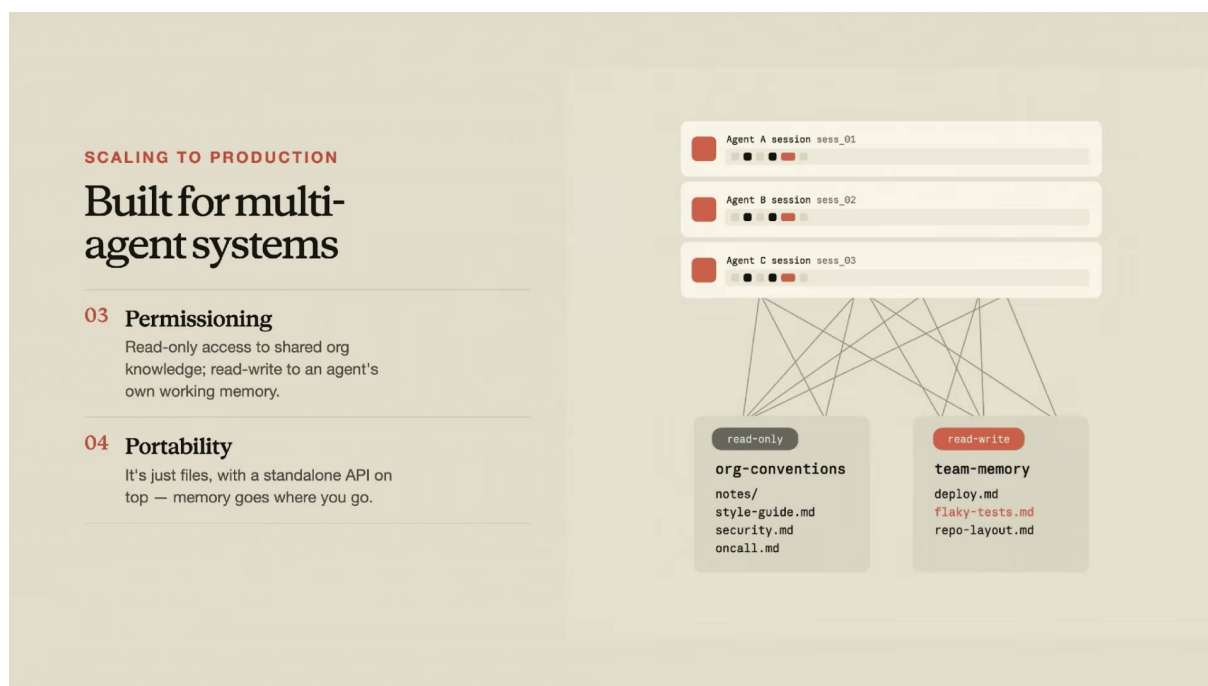


图 7: 多智能体记忆系统需要同时考虑权限和可移植性: 谁能读写哪些层级, 以及这些记忆如何跨产品界面复用。²⁵

写权限同样敏感

权限设计的常见坑是只保护“读取秘密”, 忽略“写入公共知识”。对智能体记忆而言, 写权限同样敏感, 因为错误写入会改变未来行为。组织级上下文尤其需要审批、来源记录和回滚机制。

3.5 Portability: 把记忆做成可复用资产

第四个护栏是 portability (可移植性)。如果一个团队投入大量精力整理组织、用户或任务特定上下文, 这些上下文不应被单一界面或单一智能体产品锁住。演讲者强调应通过 clean API (干净 API) 让记忆能被多个 product surfaces (产品界面) 和多个系统访问。²⁶

²⁴Source: 00:12:11–00:13:06.

²⁵视频画面时间区间: 00:12:11–00:13:51。

²⁶Source: 00:13:08–00:13:51.

可移植性的工程含义是：记忆库要有清楚的数据边界、访问接口、权限模型和版本语义。否则，团队会在每个新产品里重新积累同样的经验，形成许多互相矛盾的小记忆库。更好的做法是让被验证过的组织知识成为共享资产，然后由不同系统按权限和任务需要读取。

可移植性和权限要一起设计

可移植性和权限需要一起设计。记忆能跨系统复用，价值会升高；记忆能跨系统传播，风险也会升高。API 要暴露可控的读取和更新通道，避免把底层文件或数据库裸露给所有智能体。

3.6 Production outcomes: 护栏的目标是让自治产生收益

把四个护栏合起来，生产系统得到的结果是更稳定的自治收益，而非更花哨的记忆形式。演讲者总结了几个结果：第二次执行相似任务时准确率更高，因为智能体已经记录过之前的问题；速度和成本改善，因为任务更容易一次性完成，token 消耗更少；产品开发者也能把注意力从反复灌输上下文转向真正的产品胜利。²⁷

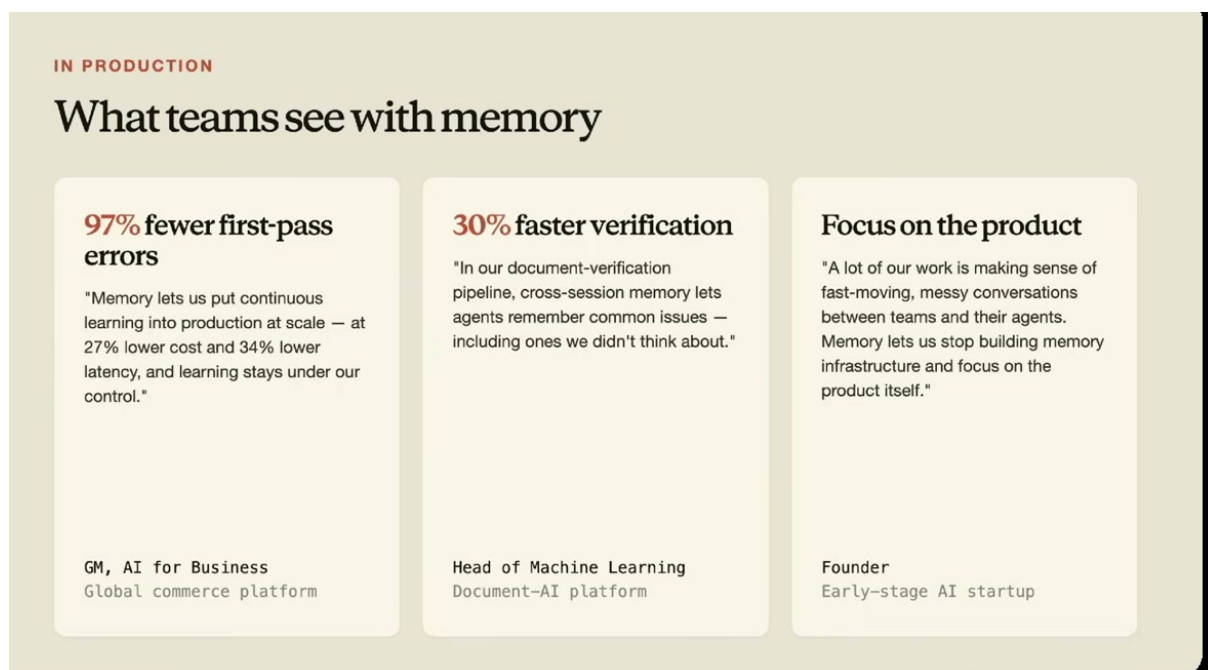


图 8: 记忆系统落地后的收益包括更少首轮错误、更快验证，以及把工程注意力从维护记忆基础设施转回产品本身。²⁸

这里需要保持清醒：这些收益依赖“基础设施已经搭好”这一前提。没有版本、哈希、权限和 API，自治写入会把短期便利换成长远混乱；有了护栏，记忆才可能成为持续学习的底座。

3.7 本章小结

第 3 节完成了从“记忆原语”到“生产系统”的升级。它的主线是：共享记忆越有价值，越需要工程治理。Versioning 让每条更新可追溯、可回滚；concurrency 用哈希检查避免覆盖并行写入；permissions 把组织级知识、团队记忆和智能体草稿分层治理；portability 让被整理过的上下文成为

²⁷Source: 00:14:07–00:15:20.

²⁸视频画面时间区间：00:14:24–00:15:31。

可复用资产。下一节会继续推进：即使这些护栏都到位，会话内记忆（in-band memory）仍然有天然边界，因为单个会话无法承担整个智能体群体的长期学习任务。

4 In-band 记忆的边界：为什么单个会话看不到系统性错误

第 4 节的结论是：in-band memory（会话内记忆）能让下一次相似任务更快受益，却无法独自承担智能体群体的持续学习目标。原因很硬：同一个会话里的 token、时间和注意力要同时服务当前任务与未来记忆整理；同一个智能体也只能看见自己的上下文窗口，无法自然发现跨会话、跨环境、跨智能体反复出现的系统性错误。

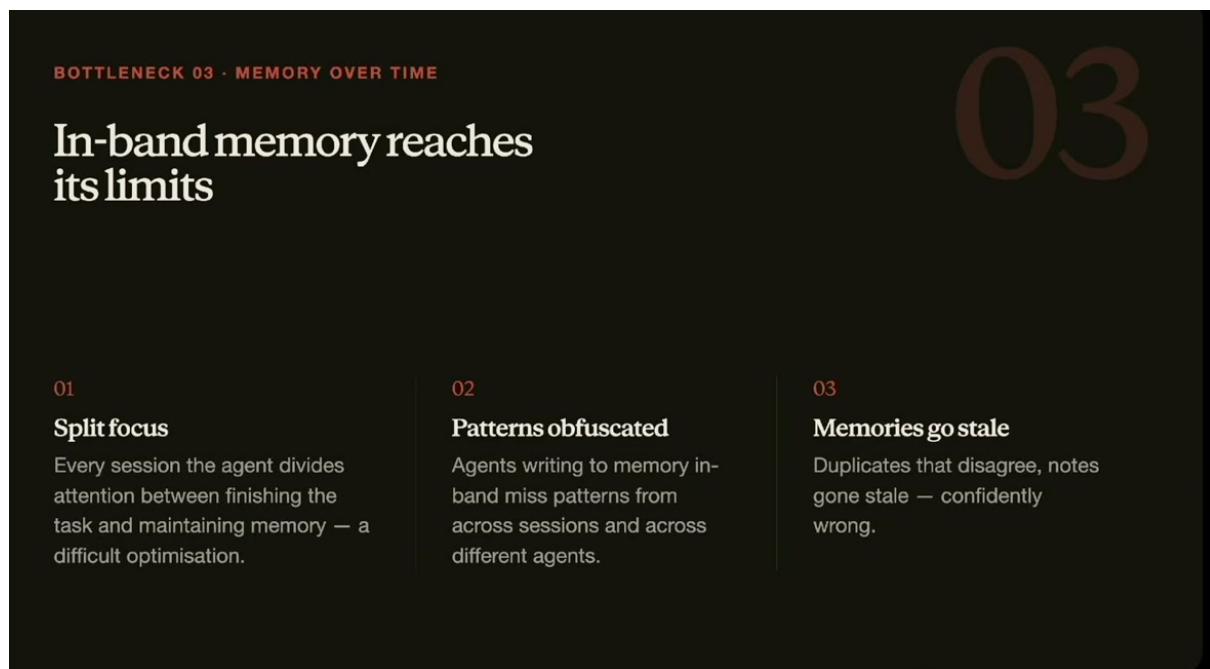


图 9: 会话内记忆的瓶颈来自三件事：注意力被任务和记忆整理分割、单个会话看不到跨会话模式、记忆会随时间过期。²⁹

In-band memory 的真实边界

In-band memory 的边界并非“记忆没用”。真正的边界在于它把任务执行和记忆整理塞进同一个会话预算里，并且让局部智能体承担全局学习职责。这个结构会天然漏掉群体层面的重复失败。

4.1 In-band memory：读写发生在同一个活跃会话内

In-band memory 指智能体在一个具体任务会话里读取和写入记忆。演讲者用 Claude Code 类产品作例子：当用户启动新会话时，智能体会围绕当前代码库、当前请求和当前上下文来决定读什么记忆、写什么经验。³⁰ 这类机制很有用，因为它把学习动作放在任务现场：刚遇到错误时记录教训，下一次相似任务就可以更快避开。

²⁹视频画面时间区间：00:15:31–00:18:10。

³⁰Source: 00:15:23–00:15:59.

问题也正来自“任务现场”。会话内智能体的首要责任是完成用户当下交给它的工作。记忆整理虽然重要，却经常是面向未来的投资。当前收益和未来收益共享同一块预算，系统很快会遇到分配难题。

In-band 的直觉

“In-band”可以理解为在同一条通信线路里完成两件事：一边处理当前任务，一边维护未来记忆。它的优点是反馈即时，缺点是什么额外整理都会占用当前会话的容量、延迟和推理注意力。

4.2 资源分裂：当前任务和未来学习争用同一预算

演讲者把第一个限制称为 split of focus and resources（注意力与资源分裂）：智能体被要求完成任务，同时还要投资于会帮助未来运行的 memory curation（记忆整理）。³¹ 这会形成一个优化问题：智能体应该把多少容量用于当前任务，多少容量用于未来的自己？

这个资源分配可以用一个轻量公式表达：

$$C_{\text{session}} = C_{\text{task}} + C_{\text{mem}}$$

符号说明：

- C_{session} ：单个会话可用的总容量，包含 token、时间、延迟预算和推理注意力。
- C_{task} ：用于完成当前任务的容量，例如读代码、调用工具、修复错误、回答用户。
- C_{mem} ：用于整理记忆的容量，例如提炼经验、检查已有记忆、写入更新。
- 公式含义：在会话内记忆模式下，当前任务和未来学习共享同一总预算，一方增加通常会压缩另一方。

资源分裂会挤压当前任务

资源分裂最容易被低估。一次漂亮的记忆写入可能让未来任务受益，可它也可能让当前任务变慢、让上下文窗口更拥挤、让智能体少检查一个关键文件。生产系统需要明确这类取舍，而不能把它交给临场感觉。

这个公式的直觉很朴素：一个学生一边参加考试，一边总结复习笔记，当然能学到东西；可是总结越细，考试时间越少。会话内智能体也一样，它可以边做边学，却很难在所有任务里都同时做到高质量执行和高质量整理。

4.3 可见性限制：单个会话看不到跨会话模式

第二个限制是 visibility limitation（可见性限制）。智能体只能看到自己当前会话里发生了什么，因此它不会自然看见“同一个错误在十个其他会话里也发生过”。演讲者特别指出，用户会因为智能体跨会话反复犯同一个错误而沮丧，可每次新会话都有新的 context window（上下文窗口），智能体本身没有这种长期挫败感。³²

³¹Source: 00:16:00–00:16:33.

³²Source: 00:16:35–00:17:05.

这也是系统性错误和局部错误的区别。局部错误像某次任务里走错一步，当前智能体可以修正；系统性错误像许多智能体在不同环境中反复走错同一类路，只有汇总多个会话的观察者才容易发现。单个会话内的记忆机制很难判断某条经验是偶发现象，还是值得写入共享记忆的普遍规律。

持续学习需要模式视野

持续学习真正需要的是“模式视野”：系统要知道某个失败是否在多个 transcript、多个工具调用、多个项目或多个智能体群体中重复出现。单个 in-band 会话只能提供局部证据。

4.4 学校类比：学生、教师和校长为什么分工

演讲者用学校作类比来解释为什么需要 out-of-band memory curation（会话外记忆整理）：学生提交大量作业，教师批改，校长或负责人查看全局情况。³³ 这个类比的力量在于它把“专门容量”和“全局视野”分开讲清楚了。学生当然能自我反思，可教育系统仍然设置教师和校长，因为学习质量需要专门的反馈者，也需要有人看到全校层面的模式。

把类比翻译回智能体系统：单个智能体像学生，负责完成具体任务；会话内记忆像学生的随手笔记，能帮助自己下一次少犯错；会话外分析过程像教师和校长，负责批量阅读作业、发现普遍薄弱点、调整课程或评分标准。这个分工的作用是给自主学习提供更高质量的反馈回路，同时保留智能体在任务现场的执行自治。

学校类比对应的系统角色

学校类比对应三层角色：学生对应任务智能体，教师对应专门的分析或评审过程，校长对应跨班级、跨课程的模式汇总者。它说明了一件实际工程事实：全局学习需要专门角色，不能全压在执行任务的智能体身上。

4.5 Freshness：过期记忆需要单独检查

第 4 节最后补上一类限制：memories go stale（记忆会过期）。曾经正确的代码路径、团队偏好、工具配置或产品原则，过一段时间后可能已经变化；某些记忆一开始就写错了；某些记忆甚至可能来自提示注入。³⁴ 因此，记忆系统需要一个检查过程，确认已经写入的内容仍然正确。

过期记忆的难点在于，它通常不会像编译错误那样立刻爆炸。它更像地图上保留了一条已经封路的旧路线：系统仍会自信地推荐它，用户也许要绕很久才发现问题。会话内智能体可以在碰到问题时修正局部记忆，可它很难系统性扫描整个记忆库，判断哪些内容已经失效。

过期记忆的危险性

过期记忆比缺少记忆更危险。缺少记忆时，智能体至少会重新探索；过期记忆会给智能体一种虚假的确定性，让它沿着旧规则快速前进，直到错误扩散到任务结果。

4.6 本章小结

第 4 节把生产护栏之后的下一个瓶颈讲清楚了：in-band memory 可以加速局部学习，却受限于会话预算、局部可见性和记忆新鲜度。资源公式 $C_{\text{session}} = C_{\text{task}} + C_{\text{mem}}$ 说明当前任务与未来整理会争用同一容量；跨会话盲区说明单个智能体看不到群体重复失败；学校类比说明专门反馈者和全

³³Source: 00:17:06–00:17:55.

³⁴Source: 00:18:00–00:18:10.

局观察者有现实必要；过期记忆问题则要求额外检查。下一节引出的 dreaming / 梦境整理，正是为这些限制提供会话外、批处理、专门资源的二阶记忆过程。

5 Dreaming: 异步整理经验的二阶记忆过程

Dreaming（梦境整理）是这场演讲的关键机制：它把持续学习从单次会话里的临场记忆，提升为会话外（out-of-band）的批处理式经验整理。它读取已有 memory store、跨会话 transcripts、tool calls、skills 和 metadata，寻找反复出现的失败或遗漏，再生成带证据的 ΔM 记忆更新提案。这个过程像学校里的“年级主任”定期审阅全班试卷：它把注意力放在整批学生是否暴露出同一个课程缺口、工具配置错误或组织偏好偏移。³⁵



图 10: Dreaming 是一个会话外的二阶记忆过程：它不抢占当前任务上下文，而是在批处理资源中专门整理和校验记忆。³⁶

5.1 三层结构：context、memory processes 与 dreaming

Lamis 把 dreaming 放在一个三层结构的顶端。第一层是 actual context，也就是智能体执行任务时引用的环境信息；第二层是 memory processes，允许智能体在会话中自主读取、管理和更新这些 context；第三层才是 dreaming，它在 batch and asynchronously 的模式下运行，并拥有独立分配的资源，用来检查记忆是否有效、是否过期、是否真正帮助智能体随时间变好。³⁷

³⁵来源区间：00:18:12–00:24:01。

³⁶视频画面时间区间：00:18:12–00:18:50。

³⁷来源区间：00:18:19–00:18:50。

Dreaming 的操作性定义

Dreaming 是一个 out-of-band 的二阶记忆过程：它处理“记忆系统是否仍然正确、完整、有效”这个系统性问题。它用批处理资源审阅 memory store 与历史 transcripts，输出带证据的记忆修改建议。

这里的“二阶”很重要。第一阶记忆解决“当前智能体下次怎么做得更好”；二阶整理解决“整个系统的记忆结构哪里需要修订”。类比到软件工程，第一阶像开发者在代码旁边写 TODO，二阶像团队在事故复盘后修改运行手册、报警规则和测试策略。

5.2 基本流程：从记忆库与 transcripts 到分析智能体

Dreaming 的基本输入有两类：一类是 existing memory store，也就是当前记忆库 M_t ；另一类是一段时间内的 sessions or transcripts，记为 $\{T_i\}$ 。系统把二者交给分析智能体，让它同时查看现有记忆与历史交互，识别哪些记忆缺失、失效、表达含糊，或无法覆盖反复出现的失败模式。³⁸

$$D(M_t, \{T_i\}_{i=1}^n) \rightarrow \{(\Delta M_j, E(\Delta M_j), s_j)\}_{j=1}^k$$

其中：

- D ：dreaming 过程，也就是会话外的批处理记忆整理函数。
- M_t ：时间 t 时的 memory store，包含现有的组织规则、用户偏好、技能说明和历史经验。
- T_i ：第 i 个 transcript 或 session 记录，包含用户消息、智能体回复、工具调用和执行结果。
- ΔM_j ：第 j 条 proposed memory update，即建议新增、修改、移动或清理的一条记忆。
- $E(\Delta M_j)$ ：支撑该更新的 evidence set，包括具体 transcript 片段、失败样例、工具错误和统计摘要。
- s_j ：该提案的强度评分，可以来自出现频率、影响范围、失败成本和置信度。

为什么公式有用

这个公式并不声称 dreaming 是一个单一数学函数。它的价值在于把流程边界说清楚：输入是当前记忆与历史会话，输出是“建议变更 + 证据 + 强度”。没有证据的记忆更新只是新的提示噪声；有证据的更新才有资格进入共享记忆。

³⁸来源区间：00:18:52–00:19:23。

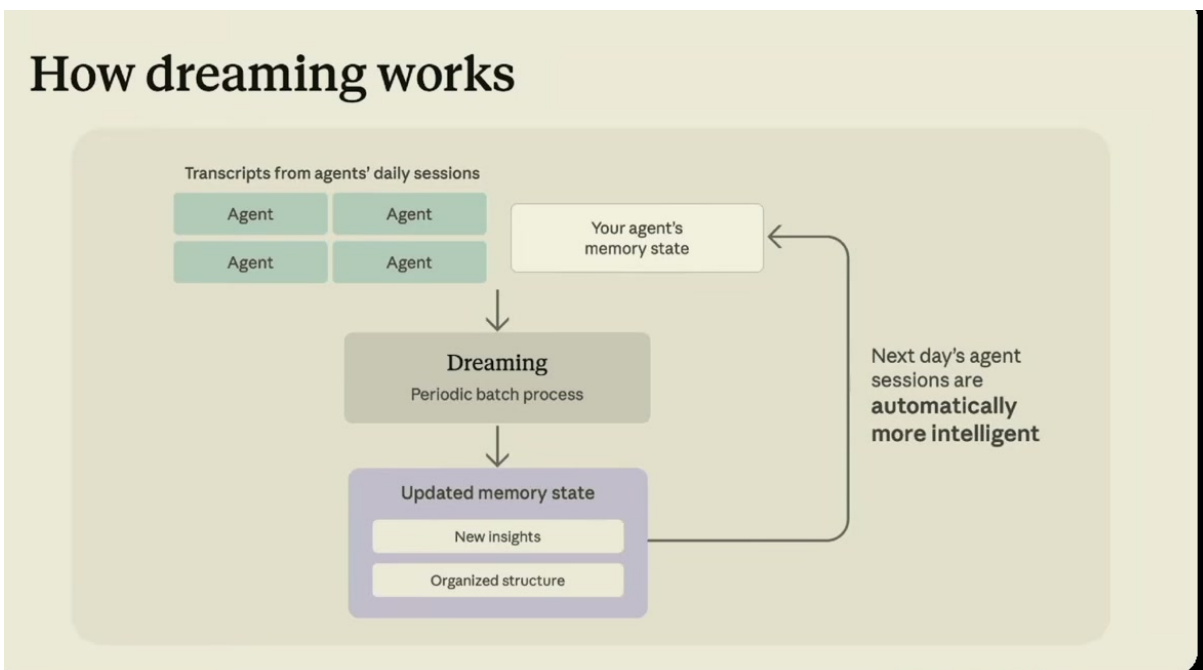


图 11: Dreaming 的基本流程是 $D(M_t, \{T_i\}) \rightarrow \{\Delta M_j, E(\Delta M_j)\}$: 用当前记忆库和一批会话转录作为输入, 产出带证据的记忆更新建议。³⁹

5.3 输出形式：带例证和流行度统计的记忆变更提案

Dreaming 的输出应是可审阅的 proposed changes。演讲中强调, 系统会提出 proposed changes, 并解释哪些 transcripts 显示了这个 pattern, 问题有多 prevalent, 以及为什么它 warrant actually updating the memory store。⁴⁰ 这让 dreaming 更像审计工作流: 它先提交变更建议, 再交给人或策略层决定接受、拒绝或细化。

好提案的三要素

一条高质量 dreaming 提案至少应包含三件事: 第一, 具体要改哪条记忆或新增哪条记忆; 第二, 哪些 transcript、tool call 或 metadata 证明这个问题真实存在; 第三, 这个问题出现得是否足够普遍, 是否值得污染共享上下文。

这种证据化输出可以降低两个风险。第一, 防止偶发失败被误写成全局规则。第二, 防止智能体根据单次偏好反馈过度概括。对组织级记忆来说, “出现过” 远远不够, 真正要问的是 “是否重复出现、是否有清晰因果、是否适合写进共享规则”。

5.4 三个直观案例：课程缺口、工具配置与组织偏好

Lamis 用学校类比给出三个案例。第一个案例是 geography students 集体答错同一道题, head teacher 发现 memory store 中缺少整个 curriculum topic, 于是建议补充课程内容。映射到智能体系统, 就是多个 agents 在相似任务中反复失败, dreaming 发现共享记忆缺少关键知识。⁴¹

³⁹ 视频画面时间区间: 00:18:52–00:21:44。

⁴⁰ 来源区间: 00:23:12–00:23:32。

⁴¹ 来源区间: 00:19:55–00:20:33。

第二个案例是数学考试里学生都把答案写成 radians，题目期望 degrees。教师由此发现计算器配置说明缺失。映射到智能体系统，就是 transcripts 中反复出现 tool calls 失败，根因可能是工具配置、参数默认值、认证方式或路径约定错误。⁴²

只看对话文本会漏掉工程根因

Dreaming 需要检查 tool calls 和 metadata。许多系统性失败藏在工具调用里，例如同一参数反复传错、同一权限范围反复被拒、同一技能反复加载失败。只读自然语言回复，会把这些错误误判成“模型表达不好”。

第三个案例是 organization-wide style preference。例如所有人都使用过多破折号，而组织希望减少这种写法。这里的重点是 fleet-wide pattern：单个用户的一次风格反馈未必值得写入组织记忆，许多会话共同显示的风格漂移才值得提出共享更新。⁴³

5.5 Orchestrator 与 subject agents：分布式审阅和聚合

生产中的 dreaming 通常需要分工。演讲描述的设计是：orchestrator（编排器）部署一批 subject agents（分析子智能体）去分析 transcripts；这些 subject agents 各自审阅分片证据，orchestrator 再汇总响应，判断哪些 patterns 足够普遍，值得提出 memory store 变更。⁴⁴

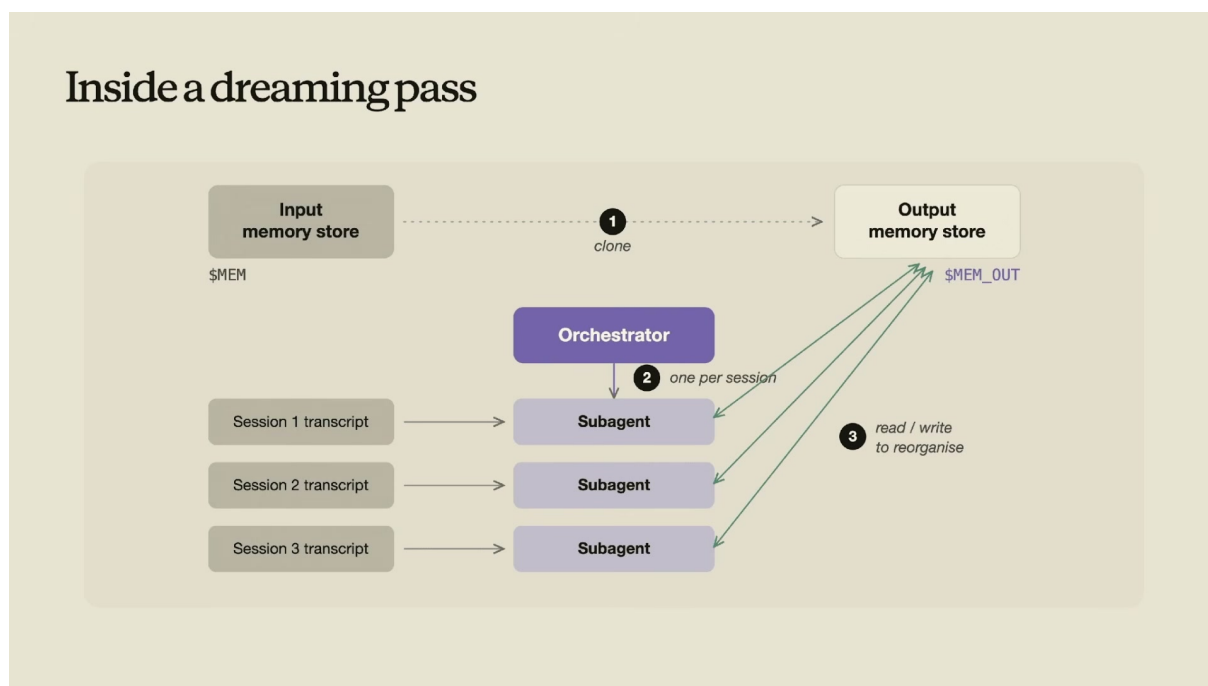


图 12: 单次 dreaming pass 把记忆库和多段会话转录交给编排器，由分析子智能体分头审阅，再汇总成可接受或拒绝的记忆更新提案。⁴⁵

这个结构解决了两个问题。第一，transcripts 规模可能很大，单个分析智能体会受到上下文窗口和注意力限制。第二，不同 subject agents 可以被 steer 到不同问题面向：有的关注工具失败，有

⁴²来源区间：00:20:33–00:21:23。

⁴³来源区间：00:21:24–00:21:44。

⁴⁴来源区间：00:22:19–00:23:12。

⁴⁵视频画面时间区间：00:22:19–00:23:35。

的关注权限边界，有的关注用户偏好，有的关注 stale memory。orchestrator 则负责把局部发现压缩成少量高置信提案。

$$O(\{A_r(T_{S_r}, M_t, G)\}_{r=1}^m) \rightarrow \{(\Delta M_j, E_j, s_j)\}_{j=1}^k$$

其中：

- O : orchestrator, 负责分派、聚合和提案筛选。
- A_r : 第 r 个 subject agent, 负责审阅一个证据切片或一个问题类型。
- T_{S_r} : 分配给第 r 个 subject agent 的 transcript 子集。
- M_t : 当前 memory store, 供 subject agents 对照“已有规则是否覆盖现实问题”。
- G : 组织给 dreaming 流程的 steering guidance, 例如哪些失败重要、哪些偏好可以忽略。
- m : subject agents 的数量。
- k : 最终输出的候选记忆更新数量。

Orchestrator 像审稿主编

subject agents 像多位审稿人，各自阅读不同证据；orchestrator 像主编，比较审稿意见，保留可验证、重复出现、影响足够大的问题。这个类比能帮助读者理解为什么 dreaming 需要聚合层，而非只让一个智能体读完所有材料。

5.6 Human or policy acceptance: 接受、拒绝或细化提案

Dreaming 的最后一步是控制权。Lamis 明确说，用户可以决定 where you want to accept changes, where you want to reject them。⁴⁶ 这句话限制了 dreaming 的权限边界：分析智能体可以提出证据化建议，最终写入共享记忆的动作仍应经过人类审阅、策略规则或产品权限模型。

这一步尤其适合引入三类结果。第一，accept：提案证据充分，影响范围明确，可以进入对应 memory scope。第二，reject：提案来自偶发情况、证据不足，或会扩大错误偏好。第三，refine：提案方向正确，措辞需要缩小范围、补充例外或移动到更合适的组织层级。

Dreaming 的危险点：把统计相关误写成组织规则

如果提案缺少接受门槛，dreaming 可能把短期噪声写成长期规则。一个夜间批处理系统一旦污染组织级 memory store，第二天许多 agents 都会继承这个错误。因此，证据、权限和审批是 dreaming 的安全阀。

5.7 本章小结

本章把 dreaming 从隐喻还原成系统流程：它是一个 out-of-band、batch、resource-dedicated 的二阶记忆过程。它读取 memory store 与 transcripts，审查 tool calls、skills 和 metadata，通过 orchestrator 与 subject agents 分布式发现模式，最后输出带证据和流行度统计的 ΔM 提案。

它解决了 in-band memory 的核心盲点：单个会话只能看到局部，dreaming 能在跨会话、跨智能体的尺度上发现重复失败。下一章会进入更现实的问题：这种机制怎样与成本、权限、enterprise APIs 和 deterministic harness 边界共同部署。

⁴⁶来源区间：00:23:35–00:23:47。

6 部署边界：成本、权限与数据库化的回归

稳健部署需要把两条学习通道同时放进系统：in-band memory 负责快速改善下一次会话，out-of-band dreaming 负责用独立资源观察更大范围的模式。真正困难的地方在部署边界：额外 token spend 是否划算，dreaming 是否严格继承权限，哪些机制应交给智能体自主处理，哪些机制应沉入 deterministic harness（执行框架 / harness）作为基础设施。Q&A 的价值正在这里：它把演讲从概念机制拉回企业系统、权限模型和数据库式原语。⁴⁷

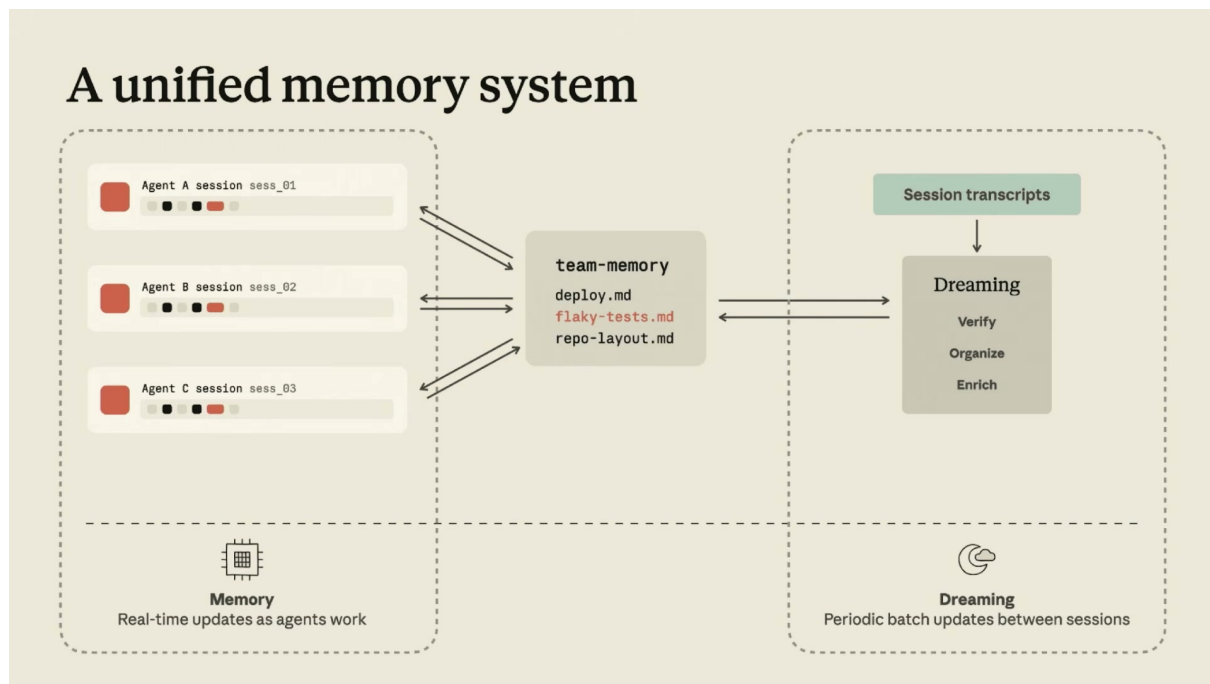


图 13: 统一记忆系统把快速的会话内写入和较慢的 dreaming 批处理并行起来：前者缩短下一次改进的时间，后者提供更宽的可见性和专门 token 预算。⁴⁸

6.1 Memory 与 dreaming 并行：快改进与广视野

Lamis 的部署图景是双通道并行。第一条通道是 memory：智能体使用 in-band context 和 in-band resources，在当前会话中把重要经验写入记忆，因此下一次 session 很快受益。第二条通道是 dreaming：它作为 out-of-band process 拥有 broader visibility 和 dedicated capacity，专门把 token spend 投向“让 agents learn better”。⁴⁹

可以用一个资源分解式表达二者差异：

$$C_{\text{total}} = C_{\text{session}} + C_{\text{dream}}, \quad C_{\text{session}} = C_{\text{task}} + C_{\text{mem}}$$

其中：

- C_{total} ：系统在一段时间内用于任务和学习的总资源。
- C_{session} ：单次会话资源，包括上下文、延迟预算和 token。

⁴⁷ 来源区间：00:23:52–00:31:39。

⁴⁸ 视频画面时间区间：00:23:52–00:25:10。

⁴⁹ 来源区间：00:23:52–00:24:48。

- C_{task} : 完成当前任务的资源。
- C_{mem} : 会话内读取、判断和写入记忆的资源。
- C_{dream} : 会话外 dreaming 批处理资源，专门用于跨会话整理和提案生成。

部署结论

in-band memory 追求短 time-to-improvement; dreaming 追求跨会话、跨智能体的 pattern visibility。二者服务同一目标，时间尺度和资源账户不同。

这也解释了为什么 Section 4 的限制没有被普通记忆系统完全解决。会话内写记忆可以让“刚发生的教训”很快被下一次使用；它仍然很难判断某类失败是否正在全组织反复出现。Dreaming 通过单独批处理弥补这个视野缺口。

6.2 成本论证：额外整理资源如何减少重复浪费

听众自然会追问：既然 dreaming 额外消耗资源，为什么值得做？Lamis 的回答是，若更好的 memory store 让 agents 更有效地 one-shot tasks，系统可能在整体上降低成本。⁵⁰ 这是一种投资回收逻辑：夜间整理多花 token，白天执行少走弯路。

$$\Delta C_{\text{net}} = C_{\text{dream}} - N \cdot p_{\text{improve}} \cdot C_{\text{retry}}$$

其中：

- ΔC_{net} : 加入 dreaming 后的净资源变化，越小越好。
- C_{dream} : 一次 dreaming 批处理成本。
- N : 后续受该记忆库影响的任务次数。
- p_{improve} : 记忆更新让任务一次成功或少重试的概率提升。
- C_{retry} : 一次失败重试、人工纠偏或工具误用带来的平均成本。

这个式子说明一个朴素事实：dreaming 不适合为了炫技而部署。它适合用于 N 足够大、失败重复性强、重试成本高、记忆更新可复用的系统。如果只是少量一次性任务，额外整理可能没有经济意义；如果是大量长期 agents，反复失败会把成本放大，整理就可能变成节省。

成本核算不能只看 token 单价

Agent 成本还包括延迟、人工介入、错误修复、权限事故、用户信任损失和工程团队排障时间。Dreaming 的回报也不能只看“少用了多少 token”，还要看是否减少重复失败与错误传播。

⁵⁰来源区间：00:24:48–00:25:10。

6.3 最终演讲总结：从简单上下文到闭环学习

在正式 Q&A 前, Lamis 给出三条总结。第一, at the very least, do the simple thing that works: 上下文管理对 agent performance 有巨大影响, CLAUDE.md、skills 和自主记忆管理已经很有价值。⁵¹ 第二, 一旦系统扩展到 many agents、long-running agents、长期工作区或复杂领域, 就要加入 safe、verifiable、auditable 的 guardrails。⁵² 第三, 如果想 close the loop, 就加入 out-of-band process like dreaming, 用来 consolidate memory、清理过期内容、补充 agents 缺失的信息并组织记忆系统。⁵³



图 14: 最后的三条实践路线是: 先做简单有效的上下文工件, 再为多智能体和长任务加入治理护栏, 最后用 dreaming 在会话之间整理和巩固记忆。⁵⁴

这套方法并不限于 coding

演讲者特别指出, memory 的适用范围覆盖 coding tasks 之外的工作。她自己也会在制作 presentations 时使用 memory, 让系统记住写作风格、幻灯片偏好等个人工作方式。这个例子提醒读者: 上下文工程的对象是“重复出现的工作环境”, 代码库只是其中一种环境。

这三条总结可以压成一个部署阶梯: 先做简单上下文, 让系统能稳定开始; 再做记忆护栏, 让多人多智能体场景可审计; 最后做 dreaming, 让系统跨会话复盘并更新记忆。它是一条渐进路线, 适合按风险和收益逐层建设。

6.4 Enterprise implementation: managed memory 与 dreaming APIs

第一段 Q&A 问的是 enterprise memory store implementation。提问者说, 个人可以把东西放在 laptop files 里, 但企业场景需要更成体系的方案。Lamis 回答时提到 managed agent solutions, 并

⁵¹ 来源区间: 00:25:14–00:25:37。

⁵² 来源区间: 00:25:38–00:26:05。

⁵³ 来源区间: 00:26:30–00:26:50。

⁵⁴ 视频画面时间区间: 00:25:14–00:27:17。

说 production memory 中的 versioning、hashing 等能力可通过 Claude Managed agents 的 memory and dreaming API 获得。⁵⁵

Q&A：企业记忆库实现

00:27:38–00:28:50

Audience: Do you have any memory store implementation that you would like to suggest?

Lamis: For a more enterprise solution, the production memory primitives like versioning and hashing are available within memory and dreaming APIs through managed agents.

这段回答需要谨慎理解。它的技术重点落在企业实现必须把 versioning、hashing、权限、审计和 dreaming API 做成可调用的基础设施。个人项目可以用 Markdown 文件和 git-like 流程起步；企业系统需要将这些原语产品化，否则共享记忆很快会遇到来源不清、覆盖冲突、权限泄漏和回滚困难。

6.5 Permission mirroring：只选择匹配权限边界的 transcripts

第二段 Q&A 触及最危险的部署问题：如果企业有 hundreds of users with different permission sets, dreaming 发生在 out-of-band context 中，如何确保它仍然遵守同样的 guardrails? Lamis 的回答是，设置 dreaming procedure 时，可以决定 exactly which session transcripts to attach，并让流程 mirror agent permissions。也就是说，系统可以只搜索与目标 memory store 拥有 same permission set 的 transcripts。⁵⁶

可以用一个权限选择公式表达：

$$\mathcal{T}_{\text{allowed}}(M_s) = \{T_i \mid P(T_i) \subseteq P(M_s) \wedge A(T_i, M_s) = 1\}$$

其中：

- M_s ：某个目标 memory scope，例如个人、团队或组织级记忆库。
- T_i ：候选 transcript。
- $P(T_i)$ ：该 transcript 的权限集合，表示它包含哪些用户、团队、项目或数据边界。
- $P(M_s)$ ：目标 memory scope 允许访问和吸收的权限集合。
- $A(T_i, M_s)$ ：额外审计谓词，例如数据保留期限、用户同意、敏感信息策略和合规标签是否满足。
- $\mathcal{T}_{\text{allowed}}(M_s)$ ：可用于该 memory scope 的 transcript 集合。

权限镜像的核心规则

Dreaming 不能默认扫描某个时间段内的一切 transcripts。安全做法是先确定目标 memory scope，再按权限、合规和审计规则选择可附加的 transcripts。记忆更新的证据边界必须小于或等于目标记忆库的访问边界。

⁵⁵来源区间：00:27:35–00:28:50。

⁵⁶来源区间：00:28:55–00:30:12。

这个约束也带来一个 unknown unknown: dreaming 生成的提案本身可能泄露证据。即使最后写入的是一条抽象规则，提案理由、例子和统计摘要也可能暴露某个团队的私有信息。因此，权限模型要同时约束三件东西：输入 transcripts、输出 ΔM 、以及提案报告 $E(\Delta M)$ 的可见范围。

6.6 Database question: 把成熟原语沉入 harness

最后一段 Q&A 问得很尖锐: versioning、concurrency、durability 这些东西听起来像是在重新发明 databases。Lamis 的回答是，需要找到边界：哪些东西可以交给 agents autonomously act，哪些应该成为 programmatic things baked into the harness。她承认 hashing、versioning 等做法正在回到成熟的软件工程原则，因为系统已经有足够信号知道这些事情应以 deterministic way 完成。⁵⁷

Q&A: 是否在重造数据库

00:30:17–00:31:39

Audience: At what point are we reinventing databases from first principles again?

Lamis: The boundary is between autonomous agent behavior and programmatic primitives baked into the harness. Once primitives like hashing and versioning prove useful, they should be codified deterministically.

这段回答给部署者一个清晰判断：智能体适合处理语义判断、证据归纳、候选更新、例外解释；harness 适合处理确定性约束，例如版本号、哈希比较、权限过滤、事务边界、审计日志和回滚。让智能体自由写 Markdown 是探索阶段；当某个原语反复证明有效，就该进入执行框架。

$$\text{Deploy}(x) = \begin{cases} \text{agent}, & x \in \{\text{semantic judgment, proposal drafting, evidence synthesis}\} \\ \text{harness}, & x \in \{\text{hashing, versioning, permissions, audit, rollback}\} \end{cases}$$

其中：

- x ：系统中的一个责任点或操作。
- agent：交给智能体处理的语义性、开放性判断。
- harness：交给 deterministic infrastructure 处理的可验证程序性约束。
- semantic judgment：例如“这类失败是否说明记忆缺失”。
- proposal drafting：例如“怎样把证据改写成一条合适的 memory update”。
- hashing、versioning、permissions：应由基础设施统一执行的稳定原语。

部署边界判断

如果一个机制要求每次都可复现、可审计、可强制执行，就不应只依赖智能体自觉遵守。哈希、版本、权限和回滚属于系统地基；智能体可以请求这些操作，harness 应负责保证它们真的发生。

⁵⁷来源区间：00:30:17–00:31:39。

6.7 本章小结

本章把 dreaming 放回真实部署场景。in-band memory 提供快速反馈, out-of-band dreaming 提供广视野和专用整理资源; 额外成本只有在减少大量重复失败时才合理; 企业实现需要把 versioning、hashing、permission mirroring 和 API 边界做成基础设施; 数据库式原语的回归说明智能体系统终究要吸收成熟软件工程经验。

最终判断很直接: 让 agents 做语义发现, 让 harness 做确定性约束。这样, memory 与 dreaming 才能从演讲概念变成可治理、可审计、可扩展的持续学习系统。

7 总结与延伸: 让智能体在睡眠中学习

最终综合的核心结论是: 让智能体在“睡眠”中学习, 工程路径应当分阶段推进。第一步用最小可行上下文让当前任务变好; 第二步用 governed memory (受治理的记忆) 让多智能体系统安全地积累经验; 第三步用 dreaming (梦境整理) 把跨会话 transcript (转录记录) 和 memory store (记忆库) 变成可审计的持续改进闭环; 第四步把权限、版本、哈希和审批边界沉入 enterprise safeguards (企业级护栏) 与 deterministic harness (确定性执行框架)。这条路线的重点在于持续改进闭环; 一次提示词变长只能改善局部表达, 证据驱动的记忆更新才会带来长期复利。⁵⁸

四级成熟度

本节的压缩判断: 智能体学习系统的成熟度可以按四级看待: minimum viable context (最小可行上下文) 解决“下一次少走弯路”; governed memory 解决“多人、多智能体、长周期共同使用时不污染共享知识”; dreaming loop 解决“跨会话发现系统性模式”; enterprise safeguards 解决“权限、审计、确定性基础设施和产品化 API”。

7.1 从演讲总结得到的建设顺序

Lamis Mukta 在结尾先给出非常务实的建议: at the very least, do the simple thing that works. 也就是说, 构建者至少应先做好 context management (上下文管理), 因为它会显著影响 agent performance (智能体表现)。她列举的起点包括 CLAUDE.md 这类上下文文件、skills (技能), 以及允许智能体自主维护这些系统。⁵⁹ 这类入口属于整个体系的第一块承重墙, 不能当成复杂平台建设前的装饰。

随后, 演讲把问题推进到规模化: 当系统里有 many agents (许多智能体)、long-running agents (长时间运行的智能体)、长期演进的 workspace/codebase (工作区或代码库), 以及 complex domains (复杂领域) 时, 记忆系统必须增加让记忆管理变得 safe, verifiable, auditable (安全、可验证、可审计) 的功能与护栏。⁶⁰ 这里的逻辑很直接: 单人草稿可以宽松, 共享组织知识必须有版本、来源、权限和回滚。

最后, 若目标是 close the loop (闭环), 系统需要增加 out-of-band process (会话外过程), 也就是演讲中称为 dreaming 的机制。它负责 consolidate memory (整合记忆)、移除已经无关的内容、补上智能体反复缺失的信息, 并清理、组织记忆系统。⁶¹ 演讲者还提醒, 这些实践不限于 coding tasks

⁵⁸Source: 00:25:14–00:27:17, 00:27:35–00:31:39.

⁵⁹Source: 00:25:18–00:25:37.

⁶⁰Source: 00:25:38–00:26:05.

⁶¹Source: 00:26:30–00:26:50.

(代码任务)：她自己在制作 presentations (演示文稿) 时也使用记忆，让系统长期学习她的写作方式和幻灯片偏好。⁶²

睡眠类比的工程含义

“睡眠中学习”只是类比，落地机制很清楚：白天的智能体会话产生任务记录、工具调用、失败轨迹和局部记忆；夜间或低峰期的批处理过程读取这些证据，提炼模式，提出记忆更新。人类睡眠会整理经验，智能体系统的 dreaming 则用工程化的 transcript review 和 memory proposal 来完成类似功能。

7.2 分阶段 Builder Checklist

下面的清单把整场演讲压缩成构建者的实施路线。它的顺序很重要：先让简单机制产生真实收益，再把治理和自动化加上去。过早建设庞大记忆平台，很容易得到一套复杂、昂贵、难以验证的系统；完全停留在单个上下文文件，又会在多智能体和企业权限场景中迅速失控。

阶段一：minimum viable context (最小可行上下文)

1. 建立一个人类可读、智能体可读的上下文入口，例如 `CLAUDE.md`、项目说明、团队偏好或任务指南。
2. 把高频流程封装为 skill (技能)，用 progressive disclosure (渐进披露) 降低上下文加载成本：先暴露名称和摘要，相关时再加载完整步骤。
3. 使用 Markdown 和普通文件系统组织初始记忆，让智能体能用 search、grep、目录浏览等基础工具定位经验。
4. 明确哪些信息值得写入：反复出现的失败、稳定偏好、代码库惯例、工具配置、审批流程，以及可复用的诊断路径。

长文件不是长期记忆系统

最小可行上下文的常见失败是把所有信息塞进一个超长文件。上下文文件越长，越容易从“导航地图”变成“噪声仓库”。更好的做法是保留清晰入口，再用技能和可搜索记忆库分层加载。

阶段二：governed memory (受治理的记忆)

1. 为 memory store 增加 versioning (版本管理)：每次更新都记录来源 transcript、会话、作者、时间和可回滚版本。
2. 为并发写入增加 hash (哈希) 检查：提交前确认 $H(M_{\text{before}}) = H(M_{\text{current}})$ ，不一致时重新加载并协调差异。
3. 为组织记忆、团队记忆、任务记忆和 scratchpad (临时工作记忆) 划分权限层级。
4. 通过 clean API (干净 API) 保持 portability (可移植性)，让整理过的记忆能服务多个产品界面和多个智能体系统。
5. 对高风险记忆更新要求证据：至少包含触发样例、影响范围、预期收益和回滚方式。

⁶²Source: 00:26:07–00:26:30.

这一步的目标是把“智能体能写记忆”升级为“智能体能在制度约束下写共享知识”。类比到城市交通：私人笔记像个人路线收藏；组织级记忆像公共路牌。公共路牌一旦写错，许多人会一起走错路，所以它需要审批、版本、责任链和修正机制。

阶段三：dreaming loop（梦境整理闭环）

1. 定期收集允许使用的 session transcripts、tool calls、metadata、已有 memory store 和失败/成功结果。
2. 让 orchestrator（编排器）把证据分发给 subject agents（分析子智能体），分别寻找缺失知识、错误配置、过期记忆和组织级偏好漂移。
3. 输出 ΔM ：候选记忆更新；同时输出 $E(\Delta M)$ ：支持该更新的证据集，包括具体 transcript 片段、发生频率和为何值得更新。
4. 由人类、策略层或产品规则决定 accept、reject、refine（接受、拒绝、细化）这些候选更新。
5. 把被接受的更新写回记忆库，并保留审计记录，以便下一轮任务和下一轮 dreaming 使用。

这个闭环可以写成轻量公式：

$$D(M_t, \{T_i\}) \rightarrow \{\Delta M_j, E(\Delta M_j)\}$$

符号说明：

- M_t ：时刻 t 的当前记忆库。
- T_i ：第 i 个会话 transcript，包含消息、工具调用和相关元数据。
- D ：dreaming 批处理过程。
- ΔM_j ：第 j 条候选记忆更新。
- $E(\Delta M_j)$ ：支持该候选更新的证据，例如重复失败样例、发生比例、影响说明和建议写入位置。

Dreaming 的价值

Dreaming 的价值不在于“更像人类做梦”这个比喻，而在于它有独立资源和更宽视野。会话内记忆追求快速改善下一次运行；dreaming 追求从大量会话中发现普遍模式，并把这些模式转化为可治理的记忆更新。

阶段四：enterprise safeguards（企业级护栏）

1. 把 memory API 和 dreaming API 设计成产品能力，而不只是后台脚本：外部系统需要稳定接口、权限模型、审计日志和失败处理。
2. 在 dreaming 中镜像权限：只把与目标 memory store 权限边界匹配的 transcripts 附加给分析过程。
3. 把已经证明有效的 primitives（原语）沉入 deterministic harness：版本、哈希、权限、回滚、审批和日志应由基础设施强制执行。

4. 明确 agent autonomy (智能体自治) 与程序化基础设施的边界: 智能体负责解释、提议和整理; harness 负责确定性约束、访问控制和提交规则。
5. 用离线评估和线上指标检查收益: 一次成功率、重复失败率、平均 token 成本、回滚次数、错误记忆扩散率和人工审批负担。

7.3 Q&A 的三个落地补充

第一, 企业实现不必从空白页开始。面对“是否有推荐的 memory storage implementation (记忆存储实现)”这一问题, Lamis 回答说, 演讲中的 production memory architecture (生产记忆架构) 对应 Anthropic managed agent solutions (托管智能体方案) 中的记忆基础设施; versioning、hashing 等能力包含在 memory and dreaming API 中。⁶³ 这段回答的技术含义是: 当记忆进入企业产品, 存储实现会自然扩展为 API、治理和运维问题, 文件格式只是其中一层。

第二, dreaming 可以遵守企业权限。问题的关键是: 若一个组织有数百个用户和不同 permission sets (权限集合), 会话外 dreaming 如何沿用用户会话中的访问边界? Lamis 的回答很具体: 设置 dreaming procedure (梦境整理流程) 时, 系统可以决定附加哪些 session transcripts, 并选择与目标 memory store 具有相同权限集合的 transcripts。⁶⁴

这可以用一个约束表达:

$$T_i \text{ may feed } M_s \text{ only if } P(T_i) \subseteq P(M_s)$$

符号说明:

- T_i : 候选 transcript。
- M_s : 某个权限范围内的 memory store。
- $P(T_i)$: 该 transcript 的访问权限集合。
- $P(M_s)$: 目标记忆库允许承载的权限集合。
- 公式含义: 只有 transcript 的权限边界被目标记忆库覆盖时, 它才可以进入对应 dreaming 过程。

权限筛选必须前置

权限安全的 dreaming 不能默认“把某段时间内所有 transcript 都扔进批处理”。可配置全量扫描是一种模式; 企业场景更常需要按用户、团队、项目、保密等级和目标记忆范围筛选证据。权限筛选应发生在 transcript 进入分析智能体之前。

第三, 数据库问题揭示了 agent autonomy 与 harness 的边界。有听众问, versioning、concurrency、durability 这些能力发展到一定程度, 是否等于重新发明 database (数据库)。Lamis 的回答把边界说清楚: 早期让智能体直接写 Markdown 并提交任意内容; 观察到哪些 primitives 确实有效后, 就应把 hashing、versioning 等能力 codify in the harness (固化到执行框架) 里, 用过去软件工程中已经有效的原则服务自治智能体。⁶⁵

⁶³Source: 00:27:35–00:28:50.

⁶⁴Source: 00:29:28–00:30:12.

⁶⁵Source: 00:30:17–00:31:39.

数据库类比的边界

数据库类比的目的在于提醒构建者：自治智能体适合处理语义判断，例如“这条经验是否值得沉淀”；确定性基础设施适合处理一致性、权限、版本和提交规则。二者配合，系统才既能学习，又不会把共享记忆交给临场发挥。

7.4 实践含义：什么时候该加 dreaming

Dreaming 需要额外 token、额外批处理资源和额外治理流程，所以它应当由问题规模触发。若一个团队只有少量短任务，CLAUDE.md、skills 和少量 Markdown memory 已经足够；若智能体开始长时间运行、跨项目工作、多用户共享组织记忆，或者反复犯相似错误，dreaming 的收益才会变得清晰。演讲者也在 00:24:48–00:25:10 解释过成本逻辑：额外的 dreaming token spend（token 开销）可能被更高的一次成功率抵消，因为智能体拥有完成任务所需的信息后，重复尝试和返工会下降。

实际落地时，可以用以下信号判断是否进入下一阶段：

- 如果智能体经常忘记项目约定，先补 minimum viable context。
- 如果多个智能体会覆盖同一份记忆，先补 governed memory。
- 如果相同失败跨会话重复出现，单个智能体又看不到全局，启动 dreaming loop。
- 如果 transcript 涉及多用户、多团队或敏感数据，把权限镜像、审计和 harness 约束放在第一批需求里。
- 如果更新后的记忆无法证明改善结果，先建设评估和证据记录，再扩大自动写入范围。

复利来自证据闭环

这套体系的商业价值来自复利：每次任务产生证据，每轮 dreaming 提炼可复用经验，每次记忆更新改善后续任务。若证据链断裂，系统只是在积累文本；若证据链闭合，系统才真正进入 continual learning（持续学习）。

7.5 开放问题：这仍是快速演化的研究与产品领域

演讲者在结尾强调，context engineering（上下文工程）中的很多进展发生在过去一年，这仍然是 open area of research and development（开放的研究与开发领域）。⁶⁶ 因此，读者应把本讲看成当前架构模式；最终教条仍不存在，工程实践还在快速变动。真正难的问题集中在以下几类：

- 记忆更新收益如何评估：应看一次成功率、用户满意度、成本、延迟，还是长期错误率？
- 证据阈值如何设定：出现几次相同失败才值得写入共享记忆？
- 过期记忆如何发现：靠定期扫描、失败触发、人工反馈，还是模型主动质疑？
- 权限边界如何验证：dreaming 的 transcript selection 是否可审计、可复现、可回滚？
- 语义判断与确定性规则如何分工：哪些更新可由智能体建议，哪些提交必须由 harness 强制检查？

⁶⁶Source: 00:26:52–00:27:17.

- 组织级偏好如何避免过拟合：少数用户的偏好何时可以升级成团队或公司级记忆？

持续学习的污染风险

最危险的未知项是“看起来学习了，实际上只是在放大偶然经验”。Dreaming 输出的每一条共享记忆都应带着证据、适用范围和撤销路径。缺少这三项，持续学习会退化持续污染。

7.6 读者自检清单

读完本讲后，读者可以用下面的短清单检查自己的智能体系统是否已经具备持续改进闭环：

1. 当前任务开始时，智能体能否快速找到最小必要上下文？
2. 高频流程是否已经从长提示词迁移到 skill 或类似的渐进加载机制？
3. 记忆库中的每条共享记忆是否知道来源、作者、时间、证据和回滚点？
4. 多个智能体同时写入时，系统是否用 hash 或等价机制检测冲突？
5. 组织级、团队级、项目级、用户级和临时 scratchpad 的读写权限是否分开？
6. Dreaming 是否只读取权限匹配的 transcripts？
7. 每条 ΔM 是否附带 $E(\Delta M)$ ，也就是具体例子、发生频率和更新理由？
8. 人类或策略层是否能 accept、reject、refine 候选记忆更新？
9. 过期记忆是否有定期检查或失败触发的刷新机制？
10. 系统是否测量了记忆更新后的真实收益，例如一次成功率、成本下降或重复失败减少？

7.7 本章小结

这场演讲最后给出的行动路线很清楚：先做简单有效的上下文管理，再把记忆系统治理起来，随后用 dreaming 把跨会话证据转化为可审批、可回滚、可审计的记忆更新，最后把成熟原语沉入企业级 harness。这样，智能体的“学习”才从单次会话里的临时聪明，变成跨任务、跨用户、跨时间的持续改进机制。